

RK628F/H HDMI-IN 功能开发指南

文件标识: RK-KF-YF-E06

发布版本: V1.1.0

日期: 2024-8-13

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供, 瑞芯微电子股份有限公司(“本公司”, 下同)不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因, 本文档将可能在未经任何通知的情况下, 不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标, 归本公司所有。

本文档可能提及的其他所有注册商标或商标, 由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴, 非经本公司书面许可, 任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部, 并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

本文档是基于RK3588/RK3576平台，使用RK628F/H开发HDMI IN功能的帮助文档。

概述

RK628F/H支持将HDMI输入信号转换成MIPI-CSI/MIPI-DSI/BT120信号输出，RK3588/RK3576支持接收该信号，实现HDMI-IN功能。本文档仅针对RK628F/H在RK3588/RK3576平台的HDMI-IN功能调试指导，其他场景可按照如下参考其他文档：

RK628F/H显示通路调试可参考：《Rockchip_Developer_Guide_RK628_For_All_Porting_CN》

RK628F/H在RK356X/RK3399等主控调试可参考：
《Rockchip_Developer_Guide_RK628_For_All_Porting_CN》

Android9/10/11版本调试可参考：《Rockchip_Developer_Guide_HDMI_IN_Based_On_CameraHal3_CN》

其他转接芯片Android12+版本调试参考：
《Rockchip_Developer_Guide_Android12+_HDMI_IN_Bridge_CN》

产品版本

芯片名称	内核版本	Android版本
RK3588	Linux 5.10/Linux 6.1	Android12/13/14
RK3576	Linux 6.1	Android14

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	范建威	2024-8-8	初始版本
V1.1.0	陈顺庆	2024-8-13	补充配置说明

目录

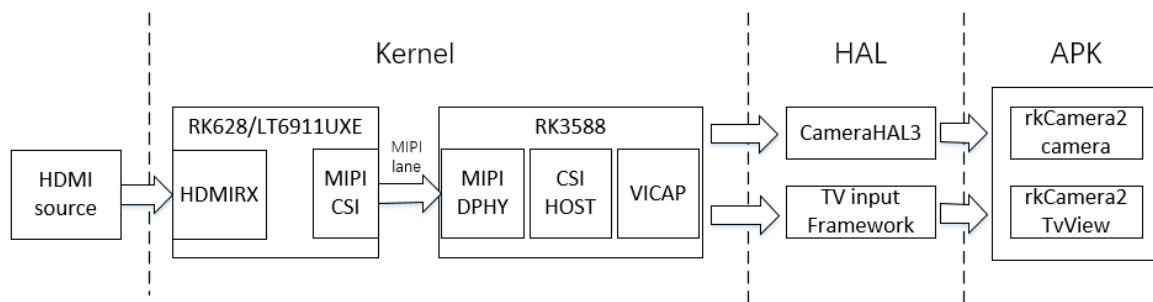
RK628F/H HDMI-IN 功能开发指南

- 1. HDMI IN功能概述
 - 1.1 RK628F/H 功能框图
 - 1.2 RK SOC接收能力
 - 1.3 RK628F/H 规格简介
- 2. HDMI IN VIDEO框架
 - 2.1 HDMIIIN VIDEO工作流程
 - 2.2 RK628F/H 主要驱动架构
 - 2.3 双MIPI拼接方案
 - 2.4 图像传输延时
- 3. 关键配置说明

- 3.1 RK628F/H DTS配置
- 3.2 Pipeline配置
- 3.3 双MIPI配置
- 3.4 CSI/DSI模式配置
- 3.5 scaler 配置
- 3.6 HDCP
- 3.7 CEC
- 3.8 低延时
 - 3.8.1 VI提前送帧机制
 - 3.8.2 使用限制
- 3.9 色域转换说明
- 4. HDMI IN APK 适配方法
 - 4.1 APK源码
 - 4.2 TV框架适配
 - 4.3 Camera框架适配
 - 4.3.1 打开宏配置
 - 4.3.2 camera3_profiles.xml配置
 - 4.3.3 切换预览
 - 4.4 TIF预览与camera预览差异
- 5. RK628F/H 使用一些限制说明
 - 5.1 特殊Timing限制
 - 5.1.1 HBP限制
 - 5.1.2 VFP限制
 - 5.1.3 Scaler限制
 - 5.2 CSI/DSI格式说明
- 6. 常用调试方法
 - 6.1 打开log开关
 - 6.2 查询驱动timings信息
 - 6.3 查看media拓扑结构
 - 6.4 查看RK628F/H HDMI-RX状态
 - 6.5 RK628F/H 设置pattern输出
 - 6.6 RK628F/H 寄存器读写
 - 6.7 HDCP调试指令
 - 6.8 开启图像数据流
 - 6.9 抓取图像文件
 - 6.10 TV预览抓图
 - 6.11 查看camera是否注册成功
- 7. 常见问题
 - 7.1 4096x2160分辨率异常问题
 - 7.2 部分源播放视频会出现断开重连
 - 7.3 信号源1440x240黑屏
 - 7.4 1366x768分辨率异常
 - 7.5 应用接口使用说明

1. HDMI IN功能概述

1.1 RK628F/H 功能框图



根据应用场景需求，RK628F/H HDMI TO MIPI-CSI2可适配TV框架（后TIF同）或是Camera框架，适配TIF框架图像传输延时更低，适配Camera框架可以使用标准Camera API，更方便录像、对接后端算法等应用功能开发。

1.2 RK SOC接收能力

RK628F/H 类似于SOC Camera设备，接入数量以及接收规格受限于MIPI PHY和DVP接口资源，同时也受限于DDR带宽。

RK3588/RK3576 HDMI转MIPI-CSI规格：

芯片型号	MIPI RX数量	最大分辨率(PHY规格)	支持格式(MIPI RX)
RK3588s	3路： 3x4lane 4路： 2x2lane + 2x4lane	DPHY: 1xDPHY-v1.2: 2.5G/lane x 4lane 4K60YUV422 DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5Gbps/lane x 4lane 4K60YUV422 (2)CPHY: 2xCPHY-v1.1: 2.5Gsp/trio(5.7Gbps) x 3trios 4K60YUV422/RGB888	YUV422/RGB888/RGB565
RK3588	4路： 4x4lane 6路： 4x2lane + 2x4lane	DPHY: 2xDPHY-v1.2: 2.5G/lane x 4lane 4K60YUV422 DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5G/lane x 4lane 4K60YUV422 (2)CPHY: 2xCPHY-v1.1: 2.5Gsp/trio(5.7Gbps) x 3trios 4K60YUV422/RGB888	YUV422/RGB888/RGB565
RK3576	3路： 3x4lane 5路： 4x2lane + 1x4lane	DPHY: 1xDPHY-v1.2: 2.5G/lane x 4lane 4K60YUV422 DCPHY: (1)DPHY: 2xDPHY-v2.0: 2.5G/lane x 4lane 4K60YUV422 (2)CPHY: 2xCPHY-v1.1: 2.5Gsp/trio(5.7Gbps) x 3trios 4K60YUV422/RGB888	YUV422/RGB888/RGB565

注：CPHY描述带宽吞吐率单位为sps，与bps的换算：1 Gsp = 2.28 Gbps。

HDMI To BT1120规格受限于PCLK大小，规格如下：

芯片型号	支持数量	最大分辨率	支持格式	逐行隔行支持
RK3588s	1	双边沿4K30	YUV422	支持逐行/隔行
RK3588	1	双边沿4K30	YUV422	支持逐行/隔行
RK3576	1	双边沿4K30	YUV422	支持逐行/隔行

1.3 RK628F/H 规格简介

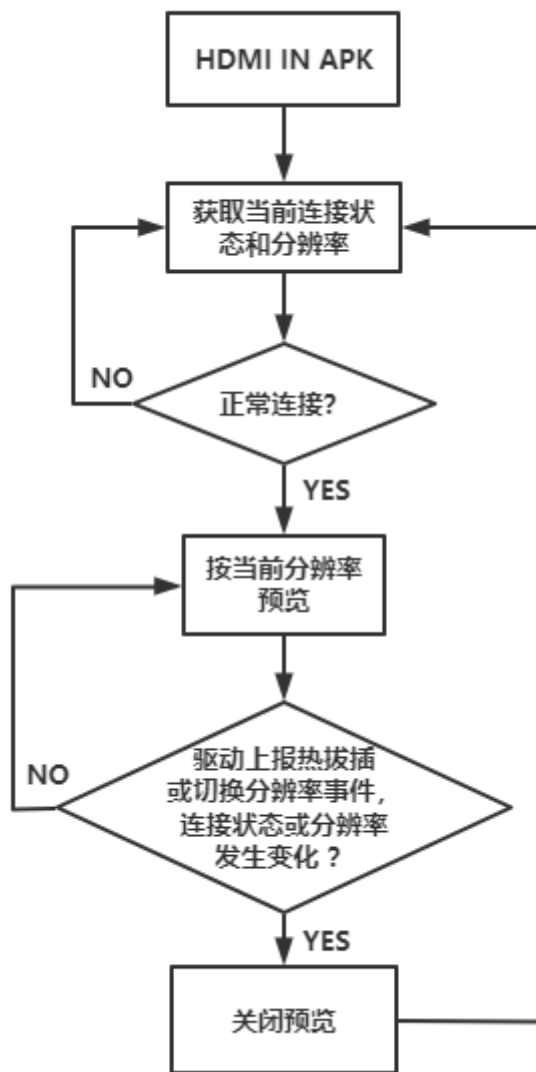
RK628F/H 规格如下表所示：

转接芯片	功能	输入接口	输出接口
RK628F	HDMI2CSI HDMI2DSI HDMI2BT1120	HDMI：4K60 RGB888 / YUV444 / YUV422 / YUV420 / YUV420 10bit	CSI: 双MIPI 4K60/YUV422 DSI: 双MIPI 4K60/RGB888 BT1120: 1080P60/YUV422

2. HDMI IN VIDEO框架

2.1 HDMI IN VIDEO工作流程

HDMI转MIPI-CSI实现HDMI IN功能流程主要如下图所示：



2.2 RK628F/H 主要驱动架构

RK628F/H驱动框架基于V4l2框架和media-framework的架构设计实现，并增加了分辨率切换功能以及热拔插功能的设计，基于V4l2/media-framework的驱动框架设计可以参考文档：

《Rockchip_Driver_Guide_VI_CN_v1.1.4.pdf》，这里重点关注分辨率变化与热拔插功能相关的几个中断和delayed_work：

- plugin_detect_irq: 5V检测中断，用于检测HDMI接口的拔插动作。
- rk628_csi_irq_handler: HDMI RX中断，监测HDMI信号变化时使用。
- delayed_work_hotplug: 5V检测中断对应的delayed_work，产生热拔插动作时，进行相应的配置处理，并设置拔插状态，上报拔插事件。
- delayed_work_res_change: HDMI RX控制器检测到信号变化中断时，对转接芯片进行重新配置，获取分辨率，上报分辨率变化事件。
- subscribe_event: 注册v4l2事件，主要注册了分辨率变化事件与拔插事件。

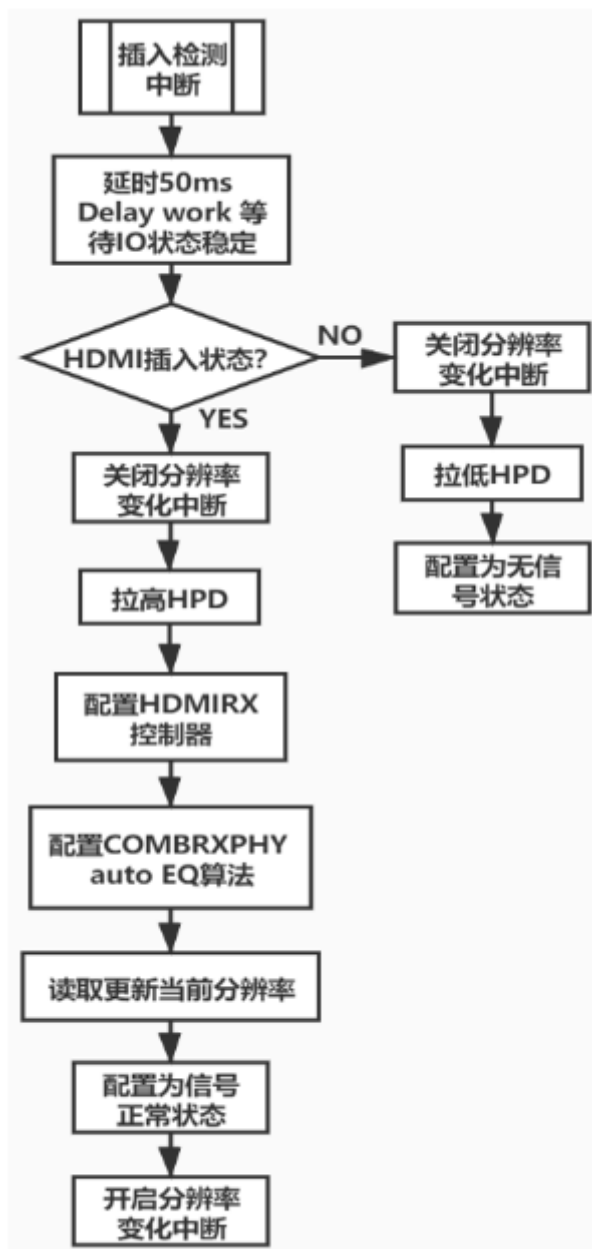
驱动主要流程参考RK628流程，主要包括：初始化、热拔插中断处理、分辨率切换中断处理。



驱动初始配置流程



分辨率变化中断处理程序



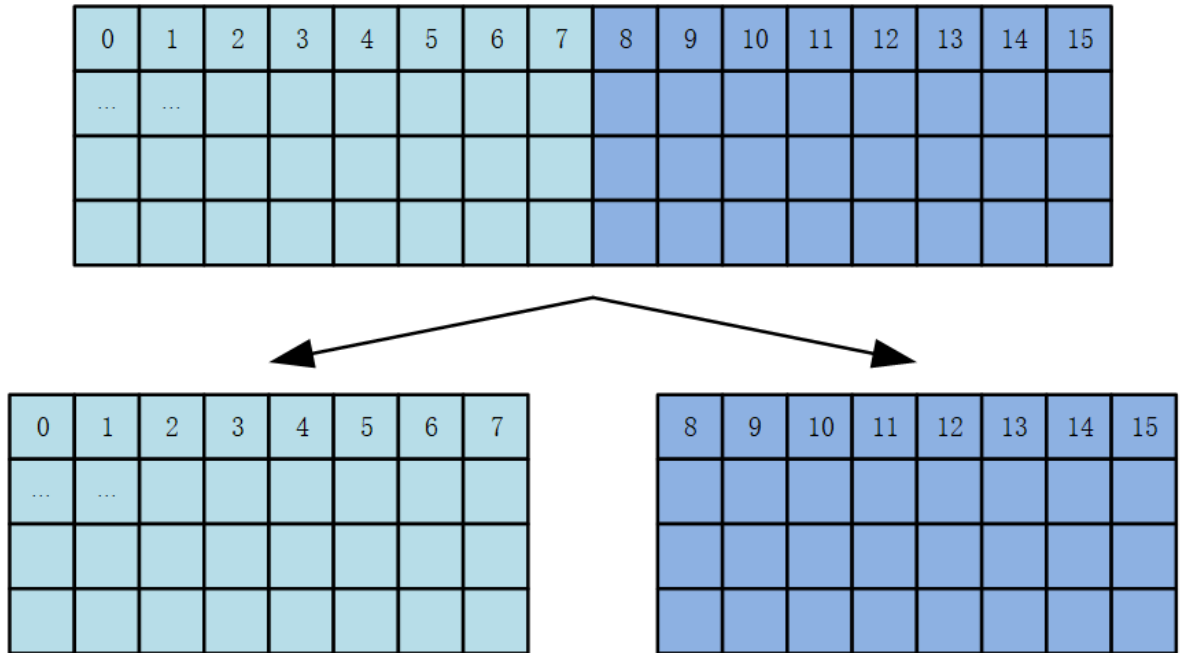
热拔插中断处理程序

2.3 双MIPI拼接方案

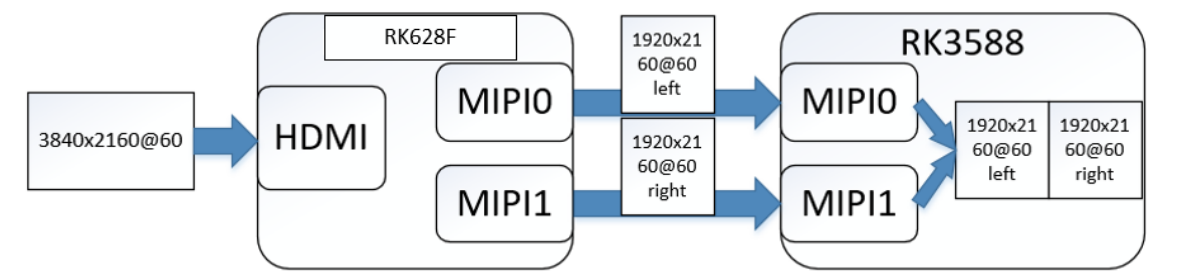
RK628F/H 受限于 MIPI DPHY 速率，一路 MIPI 最大只能支持 4K30，RK628F/H 有两路 MIPI-CSI，在对接 RK3588/RK3576 等时，4K60 的场景下，可以将 4K60 视频数据拆分成左右两半图像分别通过两路 4lane 的 MIPI-CSI/DSI 进行传输，由主控平台将图像进行合成为 4K60。

注：对接 RK3588/RK3576 SOC 平台，图像拼接合成直接在 VI 驱动完成，不会额外增加延时以及负载。

RK628F/H 内部支持可以将 HDMIRX 接收的图像 split 分割成左右两半，分别给到两路 MIPI-CSI/DSI 进行传输，如下图所示：



基于上述，在对接 RK3588/RK3576 有多路 MIPI 接口的 SOC 平台时，就可以采用双 MIPI 模式，接收下 4K60 分辨率的图像数据，功能流程如下图所示。



2.4 图像传输延时

- 对接TIF框架后加速显示，延时约为：30~50ms，叠加提前送帧低延时方案，延时约为15~30ms。
- 对接Camera框架，显示延时约为：100~120ms

3. 关键配置说明

3.1 RK628F/H DTS配置

转接芯片dts的配置如下，是i2c设备，需要配置到i2c总线下：

```
&i2c5 {
    status = "okay";
    pinctrl-0 = <&i2c5m3_xfer>;
    clock-frequency = <400000>;

    rk628_csi: rk628_csi@50 {
        reg = <0x50>;
    }
}
```

```

compatible = "rockchip,rk628-csi-v4l2";
status = "okay";
power-domains = <&power RK3576_PD_VI>;
pinctrl-names = "default";
pinctrl-0 = <&rk628_pin>;
interrupt-parent = <&gpio3>;
interrupts = <RK_PB0 IRQ_TYPE_LEVEL_HIGH>;
enable-gpios = <&gpio3 RK_PD0 GPIO_ACTIVE_HIGH>;
reset-gpios = <&gpio3 RK_PC5 GPIO_ACTIVE_HIGH>;
plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
continues-clk = <1>;
cec-enable;
#sound-dai-cells = <0>;

rockchip,camera-module-index = <0>;
rockchip,camera-module-facing = "back";
rockchip,camera-module-name = "HDMI-MIPI1";
rockchip,camera-module-lens-name = "RK628-CSI";

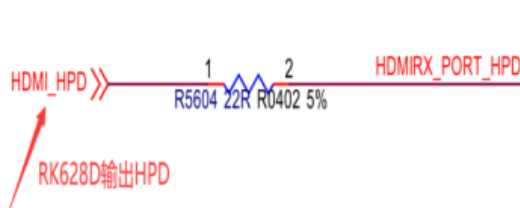
multi-dev-info {
    dev-idx-l = <1>;
    dev-idx-r = <3>;
    combine-idx = <1>;
    pixel-offset = <0>;
    dev-num = <2>;
};

port {
    hdmiin_out: endpoint {
        remote-endpoint = <&hdmi_mipi_in>;
        data-lanes = <1 2 3 4>;
    };
};
};
};

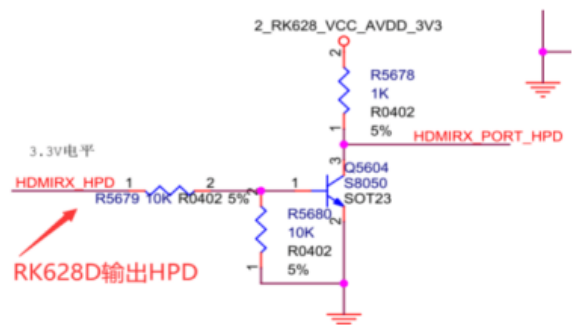
```

- interrupt-parent/ interrupts: 连接RK628中断的GPIO引脚;
- enable-gpios: RK628供电控制GPIO引脚（若为常供电可不配置）;
- reset-gpios: RK628复位控制GPIO引脚
- hpd-output-inverted: HPD输出取反配置，若HPD输出电平在电路做了取反，则需要使能此配置项;

◆ HPD未取反设计:

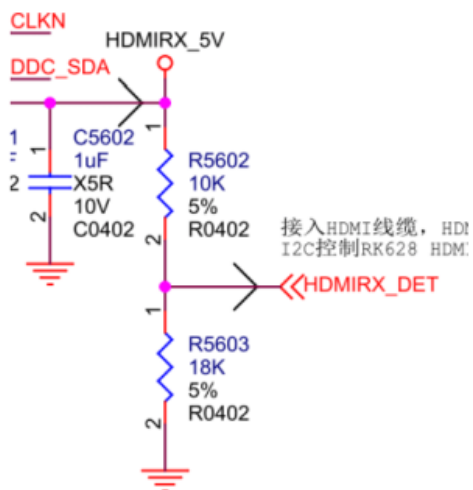


◆ HPD取反设计:

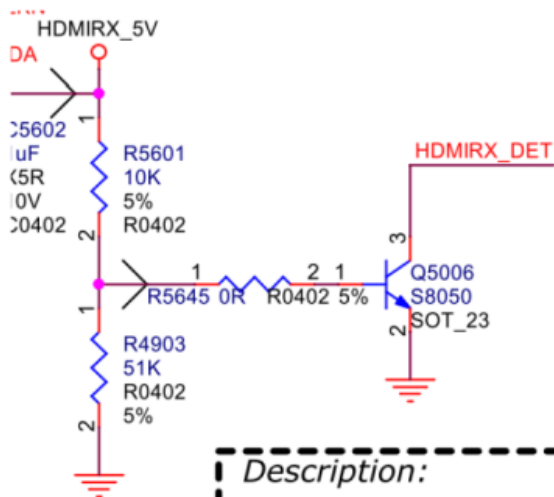


- plugin-det-gpios: HDMI插入检测GPIO引脚，注意电路是否有对电平取反，有效电平需要配置正确;

◆ HDMIRX_DET未取反设计:



◆ HDMIRX_DET取反设计:



Description:

- rockchip,camera-module-name: 对接TIF框架，该属性必须严格按照如下规律配置：HDMI-MIPIx，其中x为硬件实际接到哪个MIPI就配置哪个。RK3588按照如下规律配置。

若硬件接的是MIPI D/C PHY0，则为HDMI-MIPI0或者HDMI-MIPI

若硬件接的是MIPI D/C PHY1，则为HDMI-MIPI1

若硬件接的是MIPI CSI RX0（4lane），则为HDMI-MIPI2

若硬件接的是MIPI CSI RX0（2lane-lane01），则为HDMI-MIPI2

若硬件接的是MIPI CSI RX0（2lane-lane23），则为HDMI-MIPI3

若硬件接的是MIPI CSI RX1（4lane），则为HDMI-MIPI4

若硬件接的是MIPI CSI RX1（2lane-lane01），则为HDMI-MIPI4

若硬件接的是MIPI CSI RX1（2lane-lane23），则为HDMI-MIPI5

RK3576按照如下规律进行配置:

若硬件接的是MIPI D/C PHY，原理图上标注可能为CSI0 RX，则为HDMI-MIPI0或者HDMI-MIPI

若硬件接的是MIPI DPHY CSI1 RX（4lane），则为HDMI-MIPI1

若硬件接的是MIPI DPHY CSI1 RX（2lane: lane01 + clk0），则为HDMI-MIPI1

若硬件接的是MIPI DPHY CSI2 RX（2lane: lane23 + clk1），则为HDMI-MIPI2

若硬件接的是MIPI DPHY CSI3 RX（4lane），则为HDMI-MIPI3

若硬件接的是MIPI DPHY CSI3 RX（2lane: lane01），则为HDMI-MIPI3

若硬件接的是MIPI DPHY CSI4 RX（2lane: lane23），则为HDMI-MIPI4

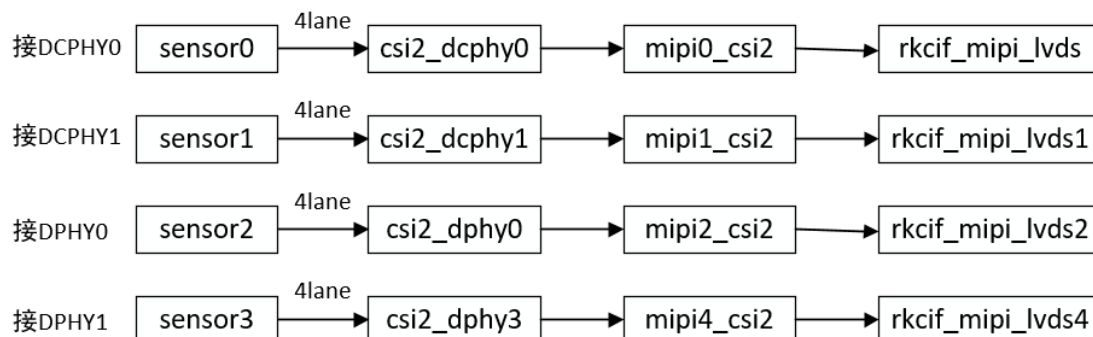
另外从DTS的pipeline链路也可以看出实际的配置，该属性与rkcif_mipi_lvds对应，rkcif_mipi_lvds对接的index是多少，则该属性也配置为多少。

- multi-dev-info: 双MIPI场景下需要增加的配置，指明左右半图如何分配，在3.3章节详细介绍。

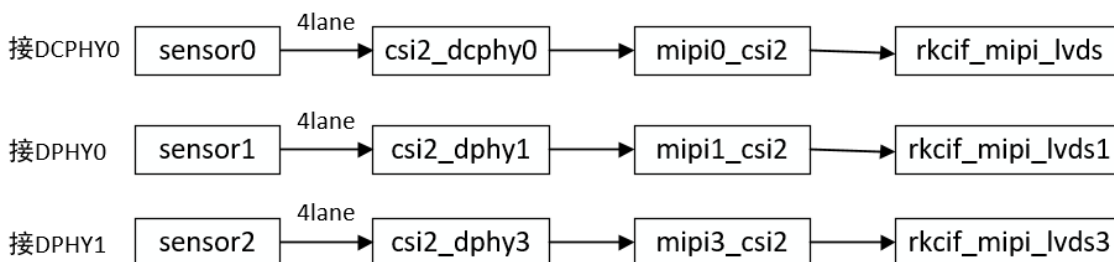
3.2 Pipeline配置

图像接收链路需要根据接的硬件接口进行配置，RK3588/RK3576有多路MIPI，需要根据实际链路进行配置。

RK3588图像链路可参考如下：



RK3576参考如下：



这里以RK628接RK3576的DPHY0为例：

```
&csi2_dphy0 {
    status = "okay";

    ports {
        #address-cells = <1>;
        #size-cells = <0>;
        port@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;

            hdmi_mipi_in: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&hdmi_in_out>;
                data-lanes = <1 2 3 4>;
            };
        };
        port@1 {
            reg = <1>;
            #address-cells = <1>;
            #size-cells = <0>;

            csidphy0_out: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&mipi1_csi2_input>;
            };
        };
    };
}
```

```

        };

    };

};

&csi2_dphy0_hw {
    status = "okay";
};

&csi2_dphy1_hw {
    status = "okay";
};

&mipil_csi2 {
    status = "okay";

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            #address-cells = <1>;
            #size-cells = <0>;

            mipil_csi2_input: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&csidphy0_out>;
            };
        };

        port@1 {
            reg = <1>;
            #address-cells = <1>;
            #size-cells = <0>;

            mipil_csi2_output: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&cif_mipi_in1>;
            };
        };
    };
};

&rkCIF {
    status = "okay";
};

&rkCIF_mipi_lvds1 {
    status = "okay";

    port {
        cif_mipi_in1: endpoint {
            remote-endpoint = <&mipil_csi2_output>;
        };
    };
};
};

```

```
&rkcif_mmu {  
    status = "okay";  
};
```

3.3 双MIPI配置

RK628F/H若采用双MIPI的方案，Pipeline链路的配置与单MIPI基本无差别，Pipeline按照RK628F/H-CSIO单MIPI的场景进行配置即可，RK628F/H的设备节点需要增加multi-dev-info的配置，解析如下：

dev-idx-l: 左半边图像对应的 mipi 接口编号

dev-idx-r: 右半边图像对应的 mipi 接口编号

combine-idx: 将左右两张图想合并到其中的一个 idx，dts 的链路以这个 idx 为准

pixel-offset: 像素偏移，默认设置 0

dev-num: 多少个 mipi 设备进行拼接

注：如果只使用一个MIPI PORT的话，DTS则不要配置multi-dev-info参数。

3.4 CSI/DSI模式配置

RK628F/H支持HDMI转MIPI-CSI或者DSI，MIPI-CSI仅支持YUV422格式输出，MIPI-DSI支持RGB888格式输出，若需要RGB888格式，则需要修改为DSI模式，DTS配置如下：

```
compatible = "rockchip,rk628-dsi-v4l2";
```

注：CSI模式仅支持YUV422输出，DSI模式仅支持RGB888输出，RGB888一般用在大屏对图像画质要求高的场景，要求RGB格式直接送显示，减少画质丢失，可走TV框架加速送显。在Camera框架下，HAL会将格式转换为NV12，会存在画质丢失。

3.5 scaler 配置

RK628F/H 支持无论HDMI什么分辨率输入都固定缩放成固定的分辨率输出，DTS打开scaler的配置，驱动默认是按照1080P60进行设置。

DTS配置：

```
scaler-en = <1>;
```

驱动固定输出分辨率，若需要其他分辨率，可自行修改。

```
static struct v4l2_dv_timings dst_timing = {  
    .type = V4L2_DV_BT_656_1120,  
    .bt = {  
        .interlaced = V4L2_DV_PROGRESSIVE,  
        .width = 1920,  
        .height = 1080,  
        .hfrontporch = 88,
```

```

        .hsync = 44,
        .hbackporch = 148,
        .vfrontporch = 4,
        .vsync = 5,
        .vbackporch = 36,
        .pixelclock = 148500000,
    },
};

```

3.6 HDCP

dts需要把hdcp-enable打开:

```

@@ -94,6 +94,7 @@ rk628_csi: rk628_csi@50 {
    plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
    continues-clk = <1>;
    cec-enable;
+   hdcp-enable;
    #sound-dai-cells = <0>;

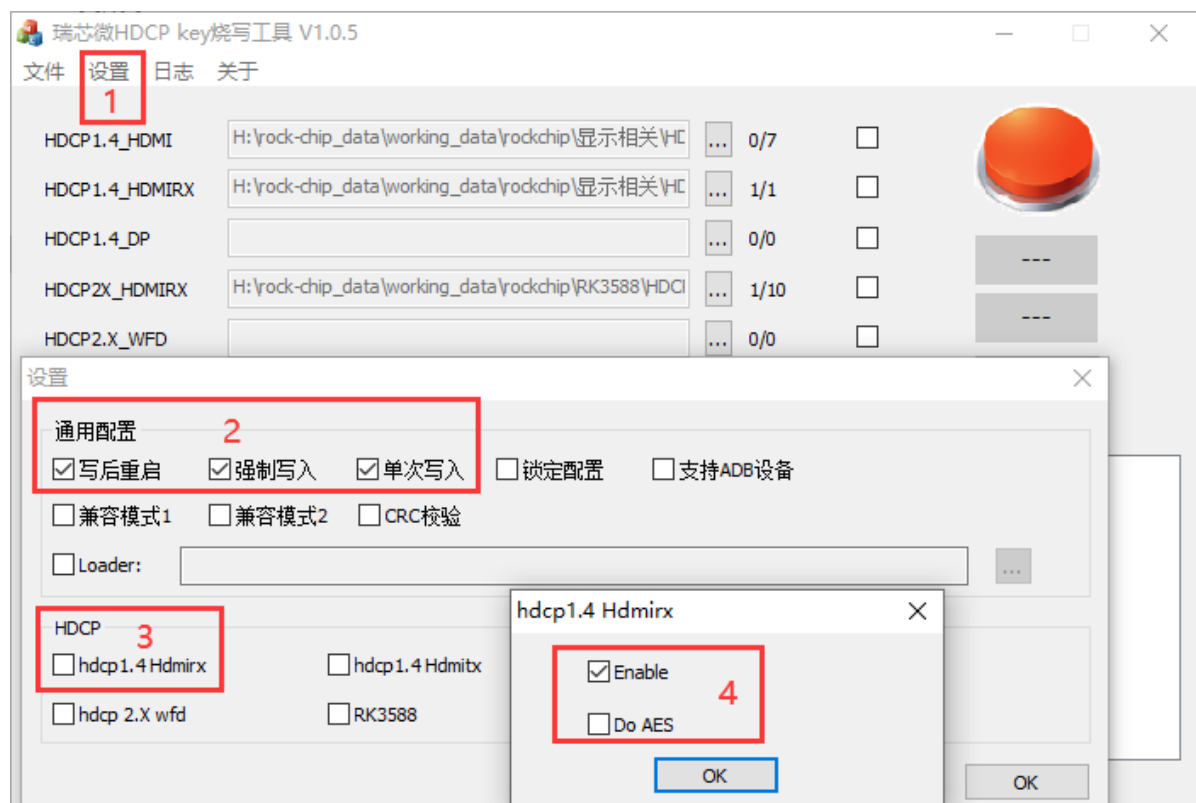
```

dts配置上hdcp-enable之后, 需要烧录HDCP1.4 RX KEY.

注: RK628只支持HDCP1.4.

烧写Key工具配置如下:

- 1、点 设置
- 2、勾选 写后重启 强制写入 单次写入
- 3、勾选 hdcp1.4 Hdmirx
- 4、勾选 Enable, 因为RK628不支持AES加密方式, 所以不要选 Do AES



HDCP的状态可以通过节点 `/d/rk628/5-0050/hdmirx/hdcp/status` 查看。

3.7 CEC

1、dts需要把cec-enable打开:

```
@@ -94,6 +94,7 @@ rk628_csi: rk628_csi@50 {
    plugin-det-gpios = <&gpio3 RK_PD5 GPIO_ACTIVE_LOW>;
    continues-clk = <1>;
+    cec-enable;
    #sound-dai-cells = <0>;
```

2、需要把CEC服务打开，并配置为TV，device/rockchip/common修改如下:

```
diff --git a/rk3576_u/BoardConfig.mk b/rk3576_u/BoardConfig.mk
index eb52fb6..2482430 100755
--- a/rk3576_u/BoardConfig.mk
+++ b/rk3576_u/BoardConfig.mk
@@ -28,4 +28,8 @@ PRODUCT_KERNEL_CONFIG += pcie_wifi.config
BOARD_GSENSOR_MXC6655XA_SUPPORT := true
BOARD_CAMERA_SUPPORT_EXT := true
BOARD_HS_ETHERNET := true
+BOARD_SUPPORT_HDMI_CEC := true
+BOARD_HDMI_CEC_TYPE := 0
```

BOARD_HDMI_CEC_TYPE的类型如下:

```
# 0      TV
# 1      Recording Device 1
# 2      Recording Device 2
# 3      Tuner 1
# 4      Playback Device 1
# 5      Audio System
# 6      Tuner 2
# 7      Tuner 3
# 8      Playback Device 2
# 9      Recording Device 3
# 10     Tuner 4
# 11     Playback Device 3
# 12     Reserved
# 13     Reserved
# 14     Specific Use
# 15     Unregistered (as Initiator address)
#        Broadcast (as Destination address)
```

注: `interrupts = <RK_PB0 IRQ_TYPE_LEVEL_HIGH>;` 中断的触发方式一定需要配置为电平触发，如果是边沿触发的话，会出现中断漏处理，CEC没有ACK导致通信异常。

3.8 低延时

3.8.1 VI提前送帧机制

如2.4所述，正常走TV框架可以将延时降低至30-50ms，在对延时更加敏感的场景，可以继续叠加VI低延时机制，将延时降低至20-30ms。设置如下节点可以开启低延时：

```
echo 100 > /sys/devices/platform/rkcif-mipi-lvds1/wait_line
echo 1 > /sys/devices/platform/rkcif-mipi-lvds1/low_latency
```

此处节点为rkcif-mipi-lvds1是因为DTS中配置到rkcif的节点是rkcif_mipi_lvds1。

3.8.2 使用限制

低延时机制在某些场景存在使用限制，目前存在如下限制：

- HDMI-IN帧率与显示帧率不匹配的场景下，低延时不生效
- RK3588平台，使能PQ之后，无法使用低延时机制
- RK3576打开PQ的DCI模块之后，低延时不生效

3.9 色域转换说明

RK628F/H CSC增加了色域、格式转换矩阵，可以根据不同客户需求，支持灵活的色域转换，目前搭配RK3576根据客户需求，最新驱动代码提供一个接口，可设置range转换的输入range，输出range均默认设置为FULL，实际效果如下表格所示。

HDMI信号源 range	用户 设置 参数	RK628F CSC 传入输入range	RK628F CSC 传入输出range	实际达成效果
Limit	Auto	Limit	Full	正常显示转换后的Full range效果
Limit	Limit	Limit	Full	正常显示转换后的Full range效果
Limit	Full	Full	Full	测试灰阶图，黑的不够黑，白的不够白 但不会并阶
Full	Auto	Full	Full	没有转换，正常显示
Full	Limit	Limit	Full	测试灰阶图，头尾的黑白并阶效果
Full	Full	Full	Full	没有转换，正常显示
Default	Auto	Full	Full	根据源的实际数据： 若信号源实际数据是Limit，则显示黑不够黑白不够白 若实际数据是Full，则正常显示
Default	Limit	Limit	Full	根据源的实际数据： 若信号源实际数据是Limit，则正常显示转换后的Full数据 若实际数据是Full，则头尾的黑白并阶效果
Default	Full	Full	Full	根据源的实际数据： 若信号源实际数据是Limit，则显示黑不够黑白不够白 若实际数据是Full，则正常显示

该设置通路，目前已经在TV框架上已经实现，打开PQ，APP可通过如下属性切换：

```
setprop persist.vendor.tvinput.rkpq.dst.range auto
setprop persist.vendor.tvinput.rkpq.dst.range limit
setprop persist.vendor.tvinput.rkpq.dst.range full
```

注：若上述range转换通路不满足客户需求，可将需求提供至redmine联系RK工程师。

4. HDMIIN APK 适配方法

4.1 APK源码

- packages/apps/TV/partner_support/samples：提供TV源数据服务，通过framework与HAL层、预览APK进行交互，由于是开机运行的隐藏服务，该APK在桌面上是隐藏图标。
- hardware/rockchip/tv_input：TVHAL层代码，开关流、热拔插和分辨率切换事件等与驱动进行命令交互。
- hardware/rockchip/camera：cameraHAL层代码，camera框架取流、拍照等功能实现。
- vendor/rockchip/hardware/interfaces/hdmi：走camera框架使用，负责监听分辨率变化与热拔插事件，与驱动以及APK交互。
- packages/apps/rkCamera2：预览apk，此应用有2个界面。默认的MainActivity界面使用TIF方案进行预览，通过framework层与上述TV源数据服务进行交互；RockchipCamera2界面使用Camera方案进行预览，用标准的Camera API，对MIPI的节点的cameraid进行open操作。
APK在桌面上图标名称为 HdmiIn，通常客户会二次开发替换为自己的预览APK。

4.2 TV框架适配

- SDK默认代码HDMI IN功能是关闭的，使能HDMI IN功能，需配置如下属性，开启后会编译含上述APK在内的相关模块：

```
vim device/rockchip/rk3588/BoardConfig.mk
BOARD_HDMI_IN_SUPPORT := true
```

APK支持RK3588 HDMI RX通路数据预览和HDMI转MIPI-CSI通路数据预览，使用时需要切换。

TIF预览方式，需要设置MIPI-CSI2通路：

```
setprop tvinput.hdmiin.type 1
```

注意：如果产品没有HDMI RX，只有RK628F/H，例如RK3576，RK3588S以及RK3588 dts禁用HDMI RX的场景，需要把tvinput.hdmiin.type属性配置到系统中，否则TV方案会黑屏无法预览RK628F/H，RK3576的举例配置如下：

```
device/rockchip/rk3576/device.mk
PRODUCT_PROPERTY_OVERRIDES += \
    tvinput.hdmiin.type=1
```

4.3 Camera框架适配

4.3.1 打开宏配置

- 走camera框架需要打开：

```
vim device/rockchip/rk3588/BoardConfig.mk
CAMERA_SUPPORT_HDMI := true
```

只配置CAMERA_SUPPORT_HDMI，而不配置BOARD_HDMI_IN_SUPPORT的情况下。如想要用rkCamera2进行camera预览，需要将rkCamera2加入到编译并配置属性persist.sys.hdmiinmode值为2。其他补充见文档：

《Rockchip_Developer_Guide_HDMI_RX_CN》

```
PRODUCT_PACKAGES += \
    rkCamera2
```

4.3.2 camera3_profiles.xml配置

走camera框架预览需要适配camera3_profiles.xml，注册camera设备。

camera3_profiles.xml文件对应SDK目录下具体芯片平台的文件：

```
hardware/rockchip/camera/etc/camera/camera3_profiles_rk3xxx.xml
```

主要配置注意事项如下，详情可参考SOC Sensor的配置方法：

- **name:** 需要与驱动名称一致，有大小写区别；
- **moduleId:** 需要与驱动dts中配置的index一致；

```
</Profiles>
<Profiles cameraId="0" name="rk628-csi" moduleId="m00">
    <Supported_hardware>
        <hwType value="SUPPORTED_HW_RKISP1"/>
    </Supported_hardware>
</Profiles>
```

```
&rk628_csi {
    status = "okay";
    /*
     * If the hpd output level is inverted on the circuit,
     * the following configuration needs to be enabled.
     */
    /* hpd-output-inverted; */
    plugin-det-gpios = <&gpio0 13 GPIO_ACTIVE_HIGH>;
    power-gpios = <&gpio0 17 GPIO_ACTIVE_HIGH>;
    rockchip,camera-module-index = <0>;
    rockchip,camera-module-facing = "back";
    rockchip,camera-module-name = "RK628-CSI";
    rockchip,camera-module-lens-name = "NC";
}
```

- scaler.availableStreamConfigurations/scaler.availableMinFrameDurations/ scaler.availableStallDurations: 需要正确配置驱动支持的分辨率和最小的帧间隔时间，若驱动中需要增加新的分辨率支持，在这里也要相应地增加配置；

```

<scaler.availableStreamConfigurations value="BLOB,3840x2160,OUTPUT,
BLOB,1920x1080,OUTPUT,
BLOB,1280x720,OUTPUT,
BLOB,720x576,OUTPUT,
BLOB,720x480,OUTPUT,
YCbCr_420_888,3840x2160,OUTPUT,
YCbCr_420_888,1920x1080,OUTPUT,
YCbCr_420_888,1280x720,OUTPUT,
YCbCr_420_888,720x576,OUTPUT,
YCbCr_420_888,720x480,OUTPUT,
IMPLEMENTATION_DEFINED,3840x2160,OUTPUT,
IMPLEMENTATION_DEFINED,1920x1080,OUTPUT,
IMPLEMENTATION_DEFINED,1280x720,OUTPUT,
IMPLEMENTATION_DEFINED,720x576,OUTPUT,
IMPLEMENTATION_DEFINED,720x480,OUTPUT" />
<scaler.availableMinFrameDurations value="BLOB,3840x2160,33333333,
BLOB,1920x1080,16666667,
BLOB,1280x720,16666667,
BLOB,720x576,20000000,
BLOB,720x480,16666667,
YCbCr_420_888,3840x2160,33333333,
YCbCr_420_888,1920x1080,16666667,
YCbCr_420_888,1280x720,16666667,
YCbCr_420_888,720x576,20000000,
YCbCr_420_888,720x480,16666667,
IMPLEMENTATION_DEFINED,3840x2160,33333333,
IMPLEMENTATION_DEFINED,1920x1080,16666667,
IMPLEMENTATION_DEFINED,1280x720,16666667,
IMPLEMENTATION_DEFINED,720x576,20000000,
IMPLEMENTATION_DEFINED,720x480,16666667"/>
<scaler.availableStallDurations value="BLOB,3840x2160,33333333,
BLOB,1920x1080,16666667,
BLOB,1280x720,16666667,
BLOB,720x576,20000000,
BLOB,720x480,16666667"/>

```

- **sensor.orientation:** 图像旋转角度，支持0、90、180、270；

```

<sensor.maxAnalogSensitivity value="2400"/> <!-- HAL
<sensor.orientation value="0"/>
<sensor.profileHueSatMapDimensions value="0,0,0"/>

```

4.3.3 切换预览

camera预览方式，需要打开RockchipCamera2界面，注意需要使能CAMERA_SUPPORT_HDMI

```
setprop persist.sys.hdmiinmode 2
```

4.4 TIF预览与camera预览差异

	TIF	Camera
优点	延迟低	app端能够拿到预览数据进行后处理
缺点	不支持屏幕旋转、分屏、画中画功能； app端拿不到预览buffer数据； 不支持screencap方式的截图命令	延迟高于TIF

1. TIF预览不支持画中画功能，要使用画中画，可以从TIF切换为camera方案。在TIF预览示例 MainActivity中，点击屏幕任一位置，弹出框的“PIP”按钮，提供了从TIF预览切换到camera画中画预览的功能，要查看该效果，前提需确保CAMERA_SUPPORT_HDMI := true，即camera预览方案是使能状态。
2. TIF预览下，支持谷歌标准的MediaProjectionManager创建虚拟屏方式进行录像与截图，也支持screenrecord命令录屏。

如果有上述录屏和截图需求，或者需要pc通过adb投屏进行投屏操作，需要配置属性：

```
vim device/rockchip/rk3588/device.mk
PRODUCT_PROPERTY_OVERRIDES += debug.sf.enable_hwc_vds=true
```

如果有重载过 device 下的产品目录，需将其配置在对应产品的目录下，可在开机后通过 adb 执行 getprop debug.sf.enable_hwc_vds 查看打印值是否为 true确认是否改动有效。

3. TIF预览下，不支持adb的screencap命令截图，包括音量键+电源键这类快捷按钮截图，需要上述第2点提到的谷歌标准MediaProjectionManager虚拟屏截图。

5. RK628F/H 使用一些限制说明

5.1 特殊Timing限制

RK628F/H 没有频点限制，但是对某些HDMIRX的特殊时序无法支持，或者需要特殊处理。

5.1.1 HBP限制

RK628F/H HBP有限制，HBP 的 寄存器只有9 bit，最大支持到 511，RK628F/H不支持HBP大于511的 timing，如果遇到该类信号源，建议通过EDID来限制（默认EDID不带这类timing，常见于使用信号发生器测试场景）。

5.1.2 VFP限制

对于部分VFP小于等于1的Timing，可能会出现画面异常现象，如：左下角出现画面花屏。

出现该现象的原因是：VFP太小，RK628F/H的vst 最小只能设置为1，相当于只有1行的缓存，但是实际前后级的时钟存在偏差，两边时钟一帧的累计偏差超过一行的话，就会出现画面花屏的异常。

出现此类问题推荐使用EDID限制的方式解决，也可以通过调节前后级的时钟来规避。

5.1.3 Scaler限制

RK628F/H 支持分辨率固定输出，内部会Scaler，但不能改变帧率，只能对图像进行缩放。
部分Timing在Scaler的时候可能会出现花屏异常，例如480P->1080P，需要微调Timing解决。

5.2 CSI/DSI格式说明

输出接口	图像格式	最大分辨率	最大MIPI速率	其他说明
CSI	YUV422	双MIPI 4K60 单MIPI 4K30	1.3Gbps/lane	
DSI	RGB888	双MIPI 4K60 单MIPI 1080P60	1.8Gbps/lane	4K60 1.8G场景需要1.2V供电

6. 常用调试方法

6.1 打开log开关

一般调试过程中建议打开debug的log开关：
打开RK628F/H 驱动debug开关，其中x为挂载的i2c，50为RK628F 的i2c地址

```
echo 2 > /sys/module/rk628_csi/parameters/debug
echo 1 > /d/rk628/x-0050/debug
dmesg -n 8
```

打开vicap驱动debug开关：

```
echo 3 > /sys/module/video_rkcif/parameters/debug
echo 8 > /proc/sys/kernel/printk
```

打开cameraHAL3 debug开关：

```
setprop persist.vendor.camera.hal.debug 5
```

打开TV debug开关

```
setprop vendor.tvinput.debug.level 3
```


6.2 查询驱动timings信息

```
rk3588_s:/ # v4l2-ctl -d /dev/v4l-subdev2 --query-dv-timings
Active width: 1920
Active height: 1080
Total width: 2200
Total height: 1125
Frame format: progressive
Polarities: -vsync -hsync
Pixelclock: 0 Hz (0.00 frames per second)
Horizontal frontporch: 88
Horizontal sync: 0
Horizontal backporch: 192
Vertical frontporch: 9
Vertical sync: 0
Vertical backporch: 36
Standards:
```

6.3 查看media拓扑结构

HDMI2MIPI驱动框架类似camera，需要保证pipeline的完整才可以正常工作，使用media-ctl查看pipeline:

```
media-ctl -d /dev/mediaX -p //X=0123...
```

以RK3588 + RK628为例，正常结果pipeline显示如下:

```
|rk3588_s:/ # media-ctl -p
Opening media device /dev/media0
Enumerating entities
Found 14 entities
Enumerating pads and links
Media controller API version 0.0.160

Media device information
-----
driver          rkcif
model           rkcif-mipi-lvds
serial
bus info
hw revision     0x0
driver version  0.0.160

Device topology
- entity 1: stream_cif_mipi_id0 (1 pad, 11 links)
    type Node subtype V4L
    device node name /dev/video0
    pad0: Sink
        <- "rockchip-mipi-csi2":1 [ENABLED]
        <- "rockchip-mipi-csi2":2 []
        <- "rockchip-mipi-csi2":3 []
        <- "rockchip-mipi-csi2":4 []
        <- "rockchip-mipi-csi2":5 []
        <- "rockchip-mipi-csi2":6 []
```

```

        <- "rockchip-mipi-csi2":7 []
        <- "rockchip-mipi-csi2":8 []
        <- "rockchip-mipi-csi2":9 []
        <- "rockchip-mipi-csi2":10 []
        <- "rockchip-mipi-csi2":11 []

- entity 5: stream_cif_mipi_id1 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video1
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 [ENABLED]
    <- "rockchip-mipi-csi2":3 []
    <- "rockchip-mipi-csi2":4 []
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 []
    <- "rockchip-mipi-csi2":7 []
    <- "rockchip-mipi-csi2":8 []
    <- "rockchip-mipi-csi2":9 []
    <- "rockchip-mipi-csi2":10 []
    <- "rockchip-mipi-csi2":11 []

- entity 9: stream_cif_mipi_id2 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video2
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 []
    <- "rockchip-mipi-csi2":3 [ENABLED]
    <- "rockchip-mipi-csi2":4 []
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 []
    <- "rockchip-mipi-csi2":7 []
    <- "rockchip-mipi-csi2":8 []
    <- "rockchip-mipi-csi2":9 []
    <- "rockchip-mipi-csi2":10 []
    <- "rockchip-mipi-csi2":11 []

- entity 13: stream_cif_mipi_id3 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video3
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 []
    <- "rockchip-mipi-csi2":3 []
    <- "rockchip-mipi-csi2":4 [ENABLED]
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 []
    <- "rockchip-mipi-csi2":7 []
    <- "rockchip-mipi-csi2":8 []
    <- "rockchip-mipi-csi2":9 []
    <- "rockchip-mipi-csi2":10 []
    <- "rockchip-mipi-csi2":11 []

- entity 17: rkCIF_scale_ch0 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video4
  pad0: Sink

```

```

        <- "rockchip-mipi-csi2":1 []
        <- "rockchip-mipi-csi2":2 []
        <- "rockchip-mipi-csi2":3 []
        <- "rockchip-mipi-csi2":4 []
        <- "rockchip-mipi-csi2":5 [ENABLED]
        <- "rockchip-mipi-csi2":6 []
        <- "rockchip-mipi-csi2":7 []
        <- "rockchip-mipi-csi2":8 []
        <- "rockchip-mipi-csi2":9 []
        <- "rockchip-mipi-csi2":10 []
        <- "rockchip-mipi-csi2":11 []

- entity 21: rkCIF_scale_ch1 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video5
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 []
    <- "rockchip-mipi-csi2":3 []
    <- "rockchip-mipi-csi2":4 []
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 [ENABLED]
    <- "rockchip-mipi-csi2":7 []
    <- "rockchip-mipi-csi2":8 []
    <- "rockchip-mipi-csi2":9 []
    <- "rockchip-mipi-csi2":10 []
    <- "rockchip-mipi-csi2":11 []

- entity 25: rkCIF_scale_ch2 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video6
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 []
    <- "rockchip-mipi-csi2":3 []
    <- "rockchip-mipi-csi2":4 []
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 []
    <- "rockchip-mipi-csi2":7 [ENABLED]
    <- "rockchip-mipi-csi2":8 []
    <- "rockchip-mipi-csi2":9 []
    <- "rockchip-mipi-csi2":10 []
    <- "rockchip-mipi-csi2":11 []

- entity 29: rkCIF_scale_ch3 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video7
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 []
    <- "rockchip-mipi-csi2":3 []
    <- "rockchip-mipi-csi2":4 []
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 []
    <- "rockchip-mipi-csi2":7 []
    <- "rockchip-mipi-csi2":8 [ENABLED]
    <- "rockchip-mipi-csi2":9 []
    <- "rockchip-mipi-csi2":10 []

```

```

        <- "rockchip-mipi-csi2":11 []

- entity 33: rkCIF_tools_id0 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video8
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 []
    <- "rockchip-mipi-csi2":3 []
    <- "rockchip-mipi-csi2":4 []
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 []
    <- "rockchip-mipi-csi2":7 []
    <- "rockchip-mipi-csi2":8 []
    <- "rockchip-mipi-csi2":9 [ENABLED]
    <- "rockchip-mipi-csi2":10 []
    <- "rockchip-mipi-csi2":11 []

- entity 37: rkCIF_tools_id1 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video9
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 []
    <- "rockchip-mipi-csi2":3 []
    <- "rockchip-mipi-csi2":4 []
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 []
    <- "rockchip-mipi-csi2":7 []
    <- "rockchip-mipi-csi2":8 []
    <- "rockchip-mipi-csi2":9 []
    <- "rockchip-mipi-csi2":10 [ENABLED]
    <- "rockchip-mipi-csi2":11 []

- entity 41: rkCIF_tools_id2 (1 pad, 11 links)
  type Node subtype V4L
  device node name /dev/video10
  pad0: Sink
    <- "rockchip-mipi-csi2":1 []
    <- "rockchip-mipi-csi2":2 []
    <- "rockchip-mipi-csi2":3 []
    <- "rockchip-mipi-csi2":4 []
    <- "rockchip-mipi-csi2":5 []
    <- "rockchip-mipi-csi2":6 []
    <- "rockchip-mipi-csi2":7 []
    <- "rockchip-mipi-csi2":8 []
    <- "rockchip-mipi-csi2":9 []
    <- "rockchip-mipi-csi2":10 []
    <- "rockchip-mipi-csi2":11 [ENABLED]

- entity 45: rockchip-mipi-csi2 (12 pads, 122 links)
  type V4L2 subdev subtype Unknown
  device node name /dev/v4l-subdev0
  pad0: Sink
    [fmt:UYVY2X8/1920x1080
    crop.bounds:(0,0)/1920x1080
    crop:(0,0)/1920x1080]
    <- "rockchip-csi2-dphy0":1 [ENABLED]

```

```
pad1: Source
-> "stream_cif_mipi_id0":0 [ENABLED]
-> "stream_cif_mipi_id1":0 []
-> "stream_cif_mipi_id2":0 []
-> "stream_cif_mipi_id3":0 []
-> "rkcif_scale_ch0":0 []
-> "rkcif_scale_ch1":0 []
-> "rkcif_scale_ch2":0 []
-> "rkcif_scale_ch3":0 []
-> "rkcif_tools_id0":0 []
-> "rkcif_tools_id1":0 []
-> "rkcif_tools_id2":0 []

pad2: Source
-> "stream_cif_mipi_id0":0 []
-> "stream_cif_mipi_id1":0 [ENABLED]
-> "stream_cif_mipi_id2":0 []
-> "stream_cif_mipi_id3":0 []
-> "rkcif_scale_ch0":0 []
-> "rkcif_scale_ch1":0 []
-> "rkcif_scale_ch2":0 []
-> "rkcif_scale_ch3":0 []
-> "rkcif_tools_id0":0 []
-> "rkcif_tools_id1":0 []
-> "rkcif_tools_id2":0 []

pad3: Source
-> "stream_cif_mipi_id0":0 []
-> "stream_cif_mipi_id1":0 []
-> "stream_cif_mipi_id2":0 [ENABLED]
-> "stream_cif_mipi_id3":0 []
-> "rkcif_scale_ch0":0 []
-> "rkcif_scale_ch1":0 []
-> "rkcif_scale_ch2":0 []
-> "rkcif_scale_ch3":0 []
-> "rkcif_tools_id0":0 []
-> "rkcif_tools_id1":0 []
-> "rkcif_tools_id2":0 []

pad4: Source
-> "stream_cif_mipi_id0":0 []
-> "stream_cif_mipi_id1":0 []
-> "stream_cif_mipi_id2":0 []
-> "stream_cif_mipi_id3":0 [ENABLED]
-> "rkcif_scale_ch0":0 []
-> "rkcif_scale_ch1":0 []
-> "rkcif_scale_ch2":0 []
-> "rkcif_scale_ch3":0 []
-> "rkcif_tools_id0":0 []
-> "rkcif_tools_id1":0 []
-> "rkcif_tools_id2":0 []

pad5: Source
-> "stream_cif_mipi_id0":0 []
-> "stream_cif_mipi_id1":0 []
-> "stream_cif_mipi_id2":0 []
-> "stream_cif_mipi_id3":0 []
-> "rkcif_scale_ch0":0 [ENABLED]
-> "rkcif_scale_ch1":0 []
-> "rkcif_scale_ch2":0 []
-> "rkcif_scale_ch3":0 []
-> "rkcif_tools_id0":0 []
```

```
-> "rkCIF_tools_id1":0 []
-> "rkCIF_tools_id2":0 []
pad6: Source
-> "stream_CIF_mipi_id0":0 []
-> "stream_CIF_mipi_id1":0 []
-> "stream_CIF_mipi_id2":0 []
-> "stream_CIF_mipi_id3":0 []
-> "rkCIF_scale_ch0":0 []
-> "rkCIF_scale_ch1":0 [ENABLED]
-> "rkCIF_scale_ch2":0 []
-> "rkCIF_scale_ch3":0 []
-> "rkCIF_tools_id0":0 []
-> "rkCIF_tools_id1":0 []
-> "rkCIF_tools_id2":0 []
pad7: Source
-> "stream_CIF_mipi_id0":0 []
-> "stream_CIF_mipi_id1":0 []
-> "stream_CIF_mipi_id2":0 []
-> "stream_CIF_mipi_id3":0 []
-> "rkCIF_scale_ch0":0 []
-> "rkCIF_scale_ch1":0 []
-> "rkCIF_scale_ch2":0 [ENABLED]
-> "rkCIF_scale_ch3":0 []
-> "rkCIF_tools_id0":0 []
-> "rkCIF_tools_id1":0 []
-> "rkCIF_tools_id2":0 []
pad8: Source
-> "stream_CIF_mipi_id0":0 []
-> "stream_CIF_mipi_id1":0 []
-> "stream_CIF_mipi_id2":0 []
-> "stream_CIF_mipi_id3":0 []
-> "rkCIF_scale_ch0":0 []
-> "rkCIF_scale_ch1":0 []
-> "rkCIF_scale_ch2":0 []
-> "rkCIF_scale_ch3":0 [ENABLED]
-> "rkCIF_tools_id0":0 []
-> "rkCIF_tools_id1":0 []
-> "rkCIF_tools_id2":0 []
pad9: Source
-> "stream_CIF_mipi_id0":0 []
-> "stream_CIF_mipi_id1":0 []
-> "stream_CIF_mipi_id2":0 []
-> "stream_CIF_mipi_id3":0 []
-> "rkCIF_scale_ch0":0 []
-> "rkCIF_scale_ch1":0 []
-> "rkCIF_scale_ch2":0 []
-> "rkCIF_scale_ch3":0 []
-> "rkCIF_tools_id0":0 [ENABLED]
-> "rkCIF_tools_id1":0 []
-> "rkCIF_tools_id2":0 []
pad10: Source
-> "stream_CIF_mipi_id0":0 []
-> "stream_CIF_mipi_id1":0 []
-> "stream_CIF_mipi_id2":0 []
-> "stream_CIF_mipi_id3":0 []
-> "rkCIF_scale_ch0":0 []
-> "rkCIF_scale_ch1":0 []
-> "rkCIF_scale_ch2":0 []
```

```

-> "rkcif_scale_ch3":0 []
-> "rkcif_tools_id0":0 []
-> "rkcif_tools_id1":0 [ENABLED]
-> "rkcif_tools_id2":0 []

pad11: Source
-> "stream_cif_mipi_id0":0 []
-> "stream_cif_mipi_id1":0 []
-> "stream_cif_mipi_id2":0 []
-> "stream_cif_mipi_id3":0 []
-> "rkcif_scale_ch0":0 []
-> "rkcif_scale_ch1":0 []
-> "rkcif_scale_ch2":0 []
-> "rkcif_scale_ch3":0 []
-> "rkcif_tools_id0":0 []
-> "rkcif_tools_id1":0 []
-> "rkcif_tools_id2":0 [ENABLED]

- entity 58: rockchip-csi2-dphy0 (2 pads, 2 links)
  type V4L2 subdev subtype Unknown
  device node name /dev/v4l-subdev1
  pad0: Sink
    [fmt:UYVY2X8/1920x1080]
    <- "m00_b_rk628-csi 5-0050":0 [ENABLED]
  pad1: Source
    -> "rockchip-mipi-csi2":0 [ENABLED]

- entity 63: m00_b_rk628-csi 5-0050 (1 pad, 1 link)
  type V4L2 subdev subtype Sensor
  device node name /dev/v4l-subdev2
  pad0: Source
    [fmt:UYVY2X8/1920x1080]
    -> "rockchip-csi2-dphy0":0 [ENABLED]

```

6.4 查看RK628F/H HDMI-RX状态

可通过如下指令读取RK628F HDMI-RX的信息，包括分辨率信息、锁定状态、拔插状态、颜色格式等等：

```

console:/ # cat /sys/kernel/debug/rk628/3-0050/hdmirx/status
status: plugin
Clk-Ch:Lock      Ch0:Lock      Ch1:Lock      Ch2:Lock
Color Format: RGB
Timing: 1920x1080p60 (2200x1125)      hfp:88  hs:45  hbp:147  vfp:4
vs:5  vbp:36
Pixel Clk: 148500000
Mode: HDMI
Color Depth: 8 bit
Color Range: Limited
Color Space: RGB

```

6.5 RK628F/H 设置pattern输出

RK628F 支持设置scaler color bar输出，具体使用命令如下：

```
echo 1 > /sys/kernel/debug/rk628/3-0050/scaler_color_bar

//pattern 模式选择
echo 2 > /sys/kernel/debug/rk628/3-0050/scaler_color_bar
```

6.6 RK628F/H 寄存器读写

RK628 寄存器调试节点如下，当前示例 RK628 接在 I2C3，地址为 0x50：

```
console:/ # ls /sys/kernel/debug/rk628/3-0050/registers/
adapter    combtxphy  csi    dsi0  gpio0  gpio2  grf      hdmirxphy
combrxphy  cru        csil   dsil  gpio1  gpio3  hdmirx
```

寄存器节点可支持读写。

读寄存器：

```
console:/ # cat /sys/kernel/debug/rk628/3-0050/registers/cru
rk628_cru:

0xc0000: 00001031 00000441 00800000 00000007
0xc0010: 00007f00
0xc0020: 00001028 00000441 00f5c28f 00000007
0xc0030: 00007f00
0xc0040: 00005019 00000541 0099735e 00000007
0xc0050: 00007f00
0xc0060: 00000015
0xc0080: 00008989 00000003 00000007 00004b80
0xc0090: 03355460 000080d7 00008004 0000003b
0xc00a0: 00002c00 00000000 00000000 00000000
0xc00b0: 00285f58 00010008 00010008 00000103
0xc00c0: 00000300 00000703 0000000b 00000008
0xc00d0: 0001000a 00000b07
0xc0180: 00000000 00000000 00000000 00000000
0xc0190: 00000000 00000000
0xc0200: 00000000 00000000 00000000 00000000
0xc0210: 00000000
```

写寄存器：

```
rk3288:/ # echo 0x000 0xffffffff > /sys/kernel/debug/rk628/3-0050/registers/grf
```


6.7 HDCP调试指令

使能HDCP:

```
echo 1 > /d/rk628/5-0050/hdmirx/hdcp/enable
```

查看HDCP的状态

```
cat /d/rk628/5-0050/hdmirx/hdcp/status
```

6.8 开启图像数据流

开启图像数据流命令如下所示，需要根据实际情况，设置video节点、分辨率、格式等信息，video节点可以按照如下方法确认：使用上述查看media拓扑命令找到转接芯片所在的media节点并查看对应的pipeline，pipeline中节点名为：stream_cif_mipi_id0所对应的video节点既是取流的节点：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-  
video=width=3840,height=2160,pixelformat='NV12' --stream-mmap=4
```

如果是DSI RGB888格式，取流命令如下：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-  
video=width=3840,height=2160,pixelformat='RGB3' --stream-mmap=4
```

6.9 抓取图像文件

可以将图像保存到文件，通过adb pull 到设备，使用7yuv，YUVview等工具软件查看：

```
v4l2-ctl --verbose -d /dev/video0 --set-fmt-  
video=width=3840,height=2160,pixelformat='NV12' --stream-mmap=3 --stream-skip=4 -  
-stream-to=/data/3840x2160_nv12.yuv --stream-count=5 --stream-poll
```

6.10 TV预览抓图

TV若在预览是出现图像异常，可以通过TV dump图像来判断是前级驱动的问题还是后级显示的问题，TV预览场景下抓图如下步骤：

```
su  
setenforce 0  
rm -rf data/system/dumpimage  
mkdir data/system/dumpimage  
chmod 777 data/system/dumpimage  
setprop vendor.hdmiin.debug.dump 1  
setprop vendor.hdmiin.debug.dumpnum 10  
setprop vendor.tvinput.debug.dump 1  
setprop vendor.tvinput.debug.dumpnum 10
```

```
半s后立马
setprop vendor.hdmiin.debug.dumpnum 0
setprop vendor.tvinput.debug.dumpnum 0
2s后
setprop vendor.hdmiin.debug.dump 0
setprop vendor.tvinput.debug.dump 0
```

执行完成后将data/system/dumpimage 文件从设备取出，使用7YUV工具进行查看。

6.11 查看camera是否注册成功

如下命令可以查看camera是否注册成功，如果没注册上，请检查camera3_profiles.xml文件。

```
dumpsys media.camera
```

7. 常见问题

取数据流等调试常见问题可参考文档：

《Rockchip_Developer_Guide_Android12+_HDMI_IN_Bridge_CN》

这里仅介绍与RK628F/H相关的调试问题

7.1 4096x2160分辨率异常问题

接RK3576平台时，4096x2160分辨率的源数据会概率性出现异常，会出现iommu的报错，导致不可恢复，可以增加如下修改，在support_mode增加最大分辨率到4096x2160.

```
@@ -274,6 +274,15 @@ static struct rkmodule_csi_dphy_param rk3588_dcphy_param = {

static const struct rk628_csi_mode supported_modes[] = {
    {
+        .width = 4096,
+        .height = 2160,
+        .max_fps = {
+            .numerator = 10000,
+            .denominator = 600000,
+        },
+        .hts_def = 4400,
+        .vts_def = 2250,
+    }, {
        .width = 3840,
        .height = 2160,
        .max_fps = {
```

7.2 部分源播放视频会出现断开重连

部分电脑的信号源，使用win自带的播放器播放视频，在源操作播放器时，例如调整窗口或者暂停等动作，会出现RK628F信号识别到中断，导致HDMI-IN画面重新开流闪一下黑屏。

可能原因是信号源的数据没变化，但是HDMI接口的AVI有变化，产生了AVI的中断导致RK628F/H 重新走了信号锁定的流程。可以增加如下修改规避此问题：

```
diff --git a/drivers/media/i2c/rk628/rk628.h b/drivers/media/i2c/rk628/rk628.h
index a35bba051390..cbb9723d667 100644
--- a/drivers/media/i2c/rk628/rk628.h
+++ b/drivers/media/i2c/rk628/rk628.h
@@ -309,6 +309,9 @@ struct rk628 {
    struct gpio_desc *hdmirx_det_gpio;
    bool last_mipi_status;
    bool is_suspend;
+   u8 video_fmt;
+   u8 color_range;
+   u8 color_space;
};

#define rk628_dbg(rk628, format, ...) \
diff --git a/drivers/media/i2c/rk628/rk628_hdmirx.c
b/drivers/media/i2c/rk628/rk628_hdmirx.c
index 388dcf38431b..88b73a007d80 100644
--- a/drivers/media/i2c/rk628/rk628_hdmirx.c
+++ b/drivers/media/i2c/rk628/rk628_hdmirx.c
@@ -1549,7 +1549,7 @@ static int rk628_hdmirx_read_timing(struct rk628 *rk628,
    u32 hofs_pix, hbp, hfp, vbp, vfp;
    u32 tmds_clk, tmdsclk_cnt;
    u64 tmp_data;
-   u8 video_fmt, vic, color_range, color_space;
+   u8 vic;

    memset(timings, 0, sizeof(struct v4l2_dv_timings));
    timings->type = V4L2_DV_BT_656_1120;
@@ -1612,10 +1612,10 @@ static int rk628_hdmirx_read_timing(struct rk628 *rk628,

    rk628_i2c_read(rk628, HDMI_RX_PDEC_AVI_PB, &val);
    vic = (val & VID_IDENT_CODE_MASK) >> 24;
-   video_fmt = rk628_hdmirx_get_format(rk628);
-   color_range = rk628_hdmirx_get_range(rk628);
-   color_space = rk628_hdmirx_get_color_space(rk628);
-   if (video_fmt == BUS_FMT_YUV420) {
+   rk628->video_fmt = rk628_hdmirx_get_format(rk628);
+   rk628->color_range = rk628_hdmirx_get_range(rk628);
+   rk628->color_space = rk628_hdmirx_get_color_space(rk628);
+   if (rk628->video_fmt == BUS_FMT_YUV420) {
        htotal *= 2;
        hact *= 2;
        hfp *= 2;
@@ -1626,7 +1626,7 @@ static int rk628_hdmirx_read_timing(struct rk628 *rk628,
    rk628_dbg(rk628, "cnt_num:%d, tmds_cnt:%d, hs_cnt:%d, vs_cnt:%d, hofs:%d\n",
        HDMIRX_MODETCLK_CNT_NUM, tmdsclk_cnt, modetclk_cnt_hs, modetclk_cnt_vs,
        hofs_pix);
```

```

    rk628_dbg(rk628, "get current aviif: vic:%d, color_range: %s, color_space
%s",
-     vic, bus_color_range_str[color_range],
bus_color_space_str[color_space]);
+     vic, bus_color_range_str[rk628->color_range], bus_color_space_str[rk628-
>color_space]);

    bt->width = hact;
    bt->height = vact;
@@ -1646,7 +1646,7 @@ static int rk628_hdmirx_read_timing(struct rk628 *rk628,
    bt->il_vsync = bt->vsync + 1;
    bt->pixelclock /= 2;
}
-   if (video_fmt == BUS_FMT_YUV420)
+   if (rk628->video_fmt == BUS_FMT_YUV420)
        bt->pixelclock *= 2;

    if (vact == 1080 && vtotal > 1500)
@@ -1867,6 +1867,9 @@ EXPORT_SYMBOL(rk628_hdmirx_scdc_ced_err);
    bool rk628_hdmirx_is_signal_change_ists(struct rk628 *rk628, u32 md_ints, u32
pdec_ints)
    {
        u32 md_mask, pded_madk;
+       u8 pre_fmt = rk628->video_fmt;
+       u8 pre_color_range = rk628->color_range;
+       u8 pre_color_space = rk628->color_space;

        md_mask = VACT_LIN_ISTS | HACT_PIX_ISTS |
            HS_CLK_ISTS | DE_ACTIVITY_ISTS |
@@ -1875,8 +1878,21 @@ bool rk628_hdmirx_is_signal_change_ists(struct rk628
*rk628, u32 md_ints, u32 pd
        return true;

        pded_madk = AVI_CKS_CHG_ISTS;
-       if (pdec_ints & pded_madk)
-           return true;
+       if (pdec_ints & pded_madk) {
+           rk628->video_fmt = rk628_hdmirx_get_format(rk628);
+           rk628->color_range = rk628_hdmirx_get_range(rk628);
+           rk628->color_space = rk628_hdmirx_get_color_space(rk628);
+           if (rk628->video_fmt != pre_fmt ||
+               rk628->color_range != pre_color_range ||
+               rk628->color_space != pre_color_space) {
+               dev_info(rk628->dev,
+                   "%s: aviinfo change! video_fmt: %s, color_range: %s,
color_space:%s\n",
+                   __func__, bus_format_str[rk628->video_fmt],
+                   bus_color_range_str[rk628->color_range],
+                   bus_color_space_str[rk628->color_space]);
+               return true;
+           }
+       }

        return false;
    }

```

7.3 信号源1440x240黑屏

因为HDMIIN分配的是Esmart图层，最大只能支持8倍的缩放，超过8倍的话无法支持，所以需要确认以下几点：

- 1、需要输出接口的分辨率，`cat /d/dri/0/summary`，确认是否是4K或是更大分辨率输出，如果是，那已经起来8倍。
- 2、如果一定要测试这么小的分辨率，可以把输出的分辨率切小一些，比如1080P。
- 3、反过来也是一样的原理，比如：IN是4K，OUT是1440x240，缩小倍数超过了8倍，一样是不支持的。

7.4 1366x768分辨率异常

RK3576/RK3588 搭配RK628F实现HDMIIN功能，使用的是RK3588/RK3576 VICAP控制器接收图像，VICAP控制器对输入图像数据有限制，必须满足宽8对齐的要求，才能正常接收，1366分辨率不满足8对齐，因此可能会出现异常，一般建议将不满足8对齐的分辨率向下载减至8对齐，也就是1366x768可裁减成1360x768，每一行会丢失6个像素。驱动可直接在`get_fmt`设置上报裁减后的分辨率，框架会自动裁剪：

```
format->format.width = ALIGN_DOWN(csi->timings.bt.width, 8);
```

若是选择走的camera框架，对应的`camera3_profiles.xml`中也必须是添加裁减后的分辨率。

7.5 应用接口使用说明

RK628F/H驱动提供部分接口供应用查询输入源的分辨率格式等信息，如应用场景需要，可以使用响应的`ioctl`调用获取驱动信息，目前已实现的`ioctl`接口包括如下：

- `VIDIOC_SUBDEV_QUERY_DV_TIMINGS`：查询信号源输入分辨率信息
- `VIDIOC_G_CTRL`：获取5V插入状态
- `VIDIOC_SUBDEV_G_FMT`：获取MIPI输出分辨率
- `RK_HDMIRX_CMD_GET_SIGNAL_STABLE_STATUS`：获取信号输入源锁定状态
- `RK_HDMIRX_CMD_GET_FPS`：获取信号源输入帧率信息
- `RK_HDMIRX_CMD_GET_INPUT_MODE`：获取输入源HDMI/DVI模式
- `RK_HDMIRX_CMD_GET_COLOR_RANGE`：获取输入源的色域信息
- `RK_HDMIRX_CMD_GET_COLOR_SPACE`：获取输入源的颜色空间
- `RK_HDMIRX_CMD_SET_COLOR_RANGE`：设置RK628F/H色域转换，效果可参考3.9色域转换说明章节

应用编程使用上述接口时，需要获取RK628F/H的设备节点，一般是`/dev/v4l-subdevX`，可以通过如下命令查看：

```
grep -H ' ' /sys/class/video4linux/v4l-subdev*/name
```

应用编程示例可参考如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <getopt.h>
#include <fcntl.h>
#include <unistd.h>
```

```

#include <sys/ioctl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <dirent.h>
#include <string.h>
#include <linux/videodev2.h>
#include <linux/v4l2-subdev.h>

#define RKMODULE_GET_HDMI_MODE \
    _IOR('V', BASE_VIDIOC_PRIVATE + 34, __u32)

#define RK_HDMIRX_CMD_GET_FPS \
    _IOR('V', BASE_VIDIOC_PRIVATE + 0, int)

#define RK_HDMIRX_CMD_GET_SIGNAL_STABLE_STATUS \
    _IOR('V', BASE_VIDIOC_PRIVATE + 1, int)

#define RK_HDMIRX_CMD_GET_COLOR_SPACE \
    _IOR('V', BASE_VIDIOC_PRIVATE + 10, int)

const int kMaxDevicePathLen = 256;
const char* kDevicePath = "/dev/";
const char kPrefix[] = "v4l-subdev";
const int kPrefixLen = sizeof(kPrefix) - 1;
const int kDevicePrefixLen = sizeof(kDevicePath) + kPrefixLen + 1;
char kV4l2DevicePath[kMaxDevicePathLen];
int mipifd = 0;

static const char *bus_color_space_str[8] = {
    "xvYCC601", "xvYCC709", "sYCC601", "Adobe_YCC601",
    "Adobe_RGB", "BT2020_YcCbCrc", "BT2020_RGB_OR_YCbCr", "RGB"
};

int find_mipi_fd()
{
    DIR *devdir;
    struct dirent* de;
    int videofd;
    int ret;

    devdir = opendir(kDevicePath);
    if(devdir == 0) {
        printf("%s: cannot open %s!\n", __FUNCTION__, kDevicePath);
        return -1;
    }

    while ((de = readdir(devdir)) != 0) {
        if (!strncmp(kPrefix, de->d_name, kPrefixLen)) {
            printf("found %s\n", de->d_name);
            snprintf(kV4l2DevicePath, kMaxDevicePathLen, "%s%s",
kDevicePath, de->d_name);
            videofd = open(kV4l2DevicePath, O_RDWR);
            if (videofd < 0){
                printf("[%s %d] open device failed:%x\n",
__FUNCTION__, __LINE__, videofd);
                continue;
            } else {
                uint32_t ishdmi;

```

```

ret = ioctl(videofd, RKMODULE_GET_HDMI_MODE,
(void*)&ishdmi);

if (ret < 0) {
    printf("RKMODULE_GET_HDMI_MODE
Failed\n");

    close(videofd);
    continue;
}
printf("%s
RKMODULE_GET_HDMI_MODE:%d\n", kV4l2DevicePath, ishdmi);
if (ishdmi) {
    mipifd = videofd;
    printf("MipiHdmi fd:%d\n", mipifd);
    if (mipifd < 0) {
        return -1;
    }
}
}
}
closedir(devdir);
return ret;
}

int main()
{
    int ret;
    int fd = -1;

    find_mipi_fd();
    fd = mipifd;
    if (fd < 0) {
        printf("find mipi dev fail\n");
        return -1;
    }

    //查询拔插状态
    struct v4l2_control control;
    control.id = V4L2_CID_DV_RX_POWER_PRESENT;
    ret = ioctl(fd, VIDIOC_G_CTRL, &control);
    if (ret < 0) {
        printf("Set POWER_PRESENT failed ,%d\n", ret);
    }

    if (control.value == 1) {
        //获取信号状态与分辨率
        int signal_status = 0;
        int ret = ioctl(fd, RK_HDMIRX_CMD_GET_SIGNAL_STABLE_STATUS,
&signal_status);
        if (ret < 0)
            printf("RK_HDMIRX_CMD_GET_SIGNAL_STABLE_STATUS failed:
%d\n", ret);

        printf("get signal status:%d\n", signal_status);
        struct v4l2_dv_timings timings;
        int err = ioctl(fd, VIDIOC_SUBDEV_QUERY_DV_TIMINGS, &timings);
        if (ret < 0)

```

```

        printf("VIDIOC_SUBDEV_QUERY_DV_TIMINGS failed: %d\n",
ret);

        const struct v4l2_bt_timings *bt = &timings.bt;

        printf("hact:%d, hfp:%d, hs:%d, hbp:%d, vact:%d, vfp:%d, vs:%d,
vbp:%d, interlace:%d\n",
        bt->width, bt->hfrontporch, bt->hsync, bt->hbackporch, bt->height, bt-
>vfrontporch, bt->vsync,
        bt->vbackporch, bt->interlaced);

        struct v4l2_subdev_format aFormat;
        err = ioctl(fd, VIDIOC_SUBDEV_G_FMT, &aFormat);
        if (err < 0)
            printf("VIDIOC_SUBDEV_G_FMT failed: %d\n", err);

        int fps;
        err = ioctl(fd, RK_HDMIRX_CMD_GET_FPS, &fps);
        if (err < 0)
            printf("RK_HDMIRX_CMD_GET_FPS failed: %d\n", err);
        printf("VIDIOC_SUBDEV_G_FMT: width: %d, height: %d, fps: %d\n",
aFormat.format.width, aFormat.format.height, fps);

        int color_space = 0;
        ret = ioctl(fd, RK_HDMIRX_CMD_GET_COLOR_SPACE, &color_space);
        if (ret < 0)
            printf("RK_HDMIRX_CMD_GET_COLOR_SPACE failed: %d\n",
ret);

        printf("get color space:%s\n", bus_color_space_str[color_space]);
    } else {
        printf("get 5V:%d, hdmi plug out\n", control.value);
    }

    close(fd);
    return 0;
}

```